HR Roster Sync

Aditya Prakash

MoveInSync Backend Case Study 5, Advanced Track

Keeping employee records correct when HR changes arrive twice, arrive late, or go missing from a nightly export.

Live console: rostersync-aditya.duckdns.org

Source code: github.com/ad8ya089/roster-sync

Why roster synchronisation needs safeguards

Duplicate delivery, late arrival, invalid rows and incomplete exports need different responses.

Duplicate delivery

Same key, same content

DUPLICATE

Late arrival

Version 7 arrives before 6

STALE

Bad record

Bad phone or unknown office

REJECTED

Short export

120,000 of about 200,000

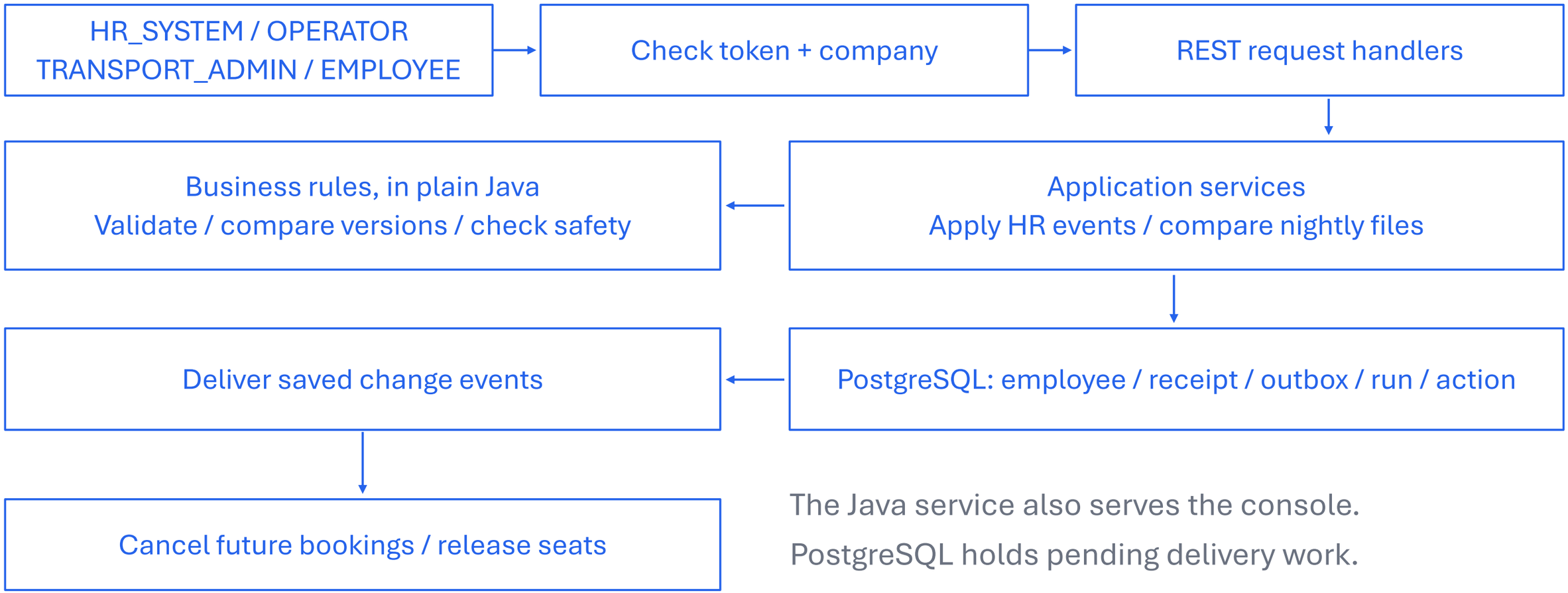
AWAITING_APPROVAL

Scale target: 200,000 employees. Complete the diff in under 60 seconds.

Tests and implementation references are in the slide notes.

How the application is organised

One Java service handles requests and background work. PostgreSQL holds durable state.



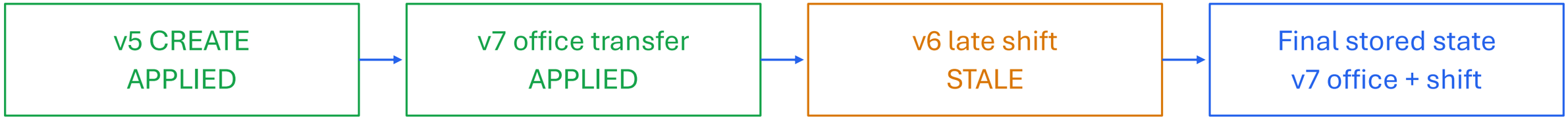
The Java service also serves the console.

PostgreSQL holds pending delivery work.

Tests and implementation references are in the slide notes.

Late events cannot overwrite newer employee state

The source version decides which state wins. Arrival time does not.



Incoming vs stored	Content	Outcome
Higher	Changed	APPLIED
Higher	Identical	NO_CHANGE, advance version
Lower	Any	STALE
Equal	Same / different	DUPLICATE / CONFLICT

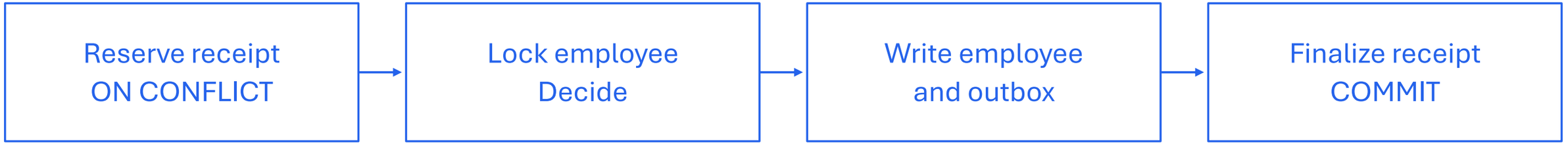
Two fixed-seed properties run 1,000 random trials each.

Tests and implementation references are in the slide notes.

How replay is detected

A database receipt remembers the first accepted content for each employee version.

Idempotency key = (tenantId, employeeId, version)



Case	Durable result
Same accepted key + same payload hash	DUPLICATE with originalOutcome
Same accepted key + different type/content	CONFLICT, audit only
First transaction rolls back	Waiting insert can succeed

Hash excludes eventId and occurredAt. Employee, receipt and outbox share one transaction.

Tests and implementation references are in the slide notes.

Why an unchanged event must still advance the version

Keeping the highest source version prevents an older event from restoring an inactive employee.

Before the fix



After the fix



Version 8 blocks the late create. A genuine rehire at version 9 still applies.

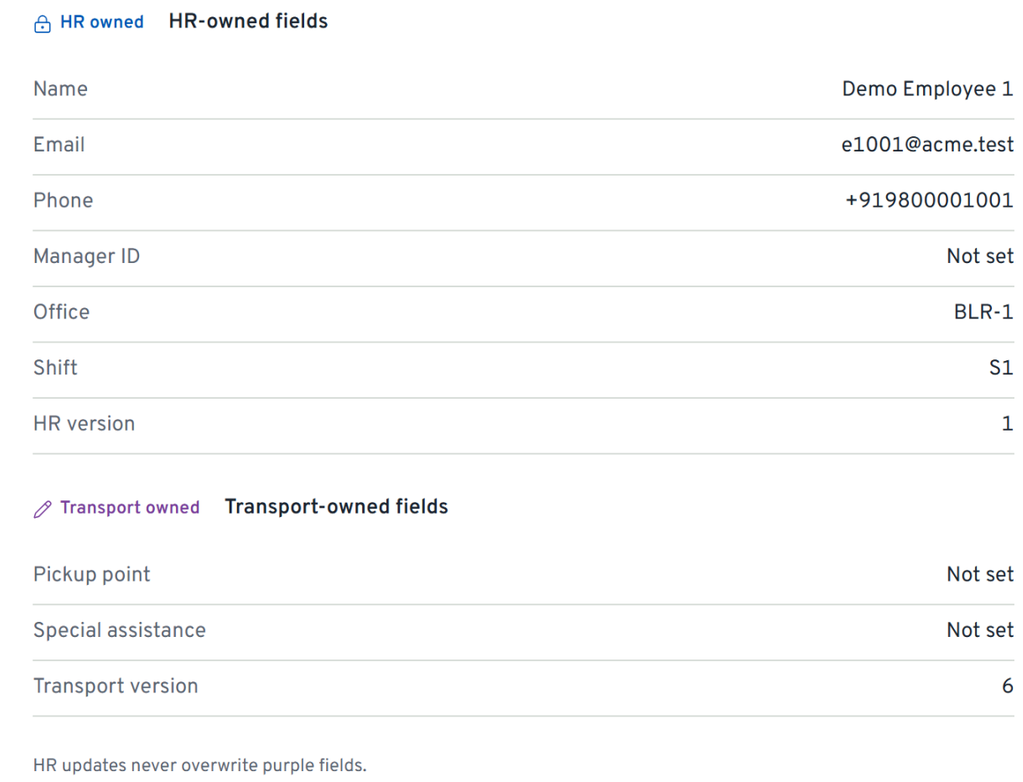
Tests and implementation references are in the slide notes.

HR and transport control different fields

An HR update cannot replace a pickup point maintained by the transport team.

HR owns	Transport owns
name, email, phone	pickupPoint
manager, office, shift, status	specialAssistance

HR input for transport fields is ignored with FIELD_NOT_OWNED_BY_SOURCE; transport PATCH rejects HR keys.



Actual employee drawer, synthetic fixture

Tests and implementation references are in the slide notes.

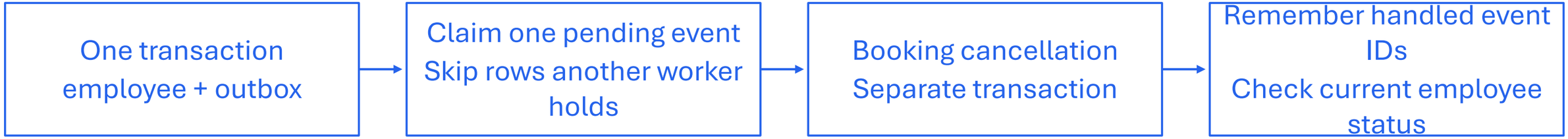
What happens if the service crashes?

The employee change and pending delivery record share the same database transaction.

Saving state and sending an event separately



The implemented path

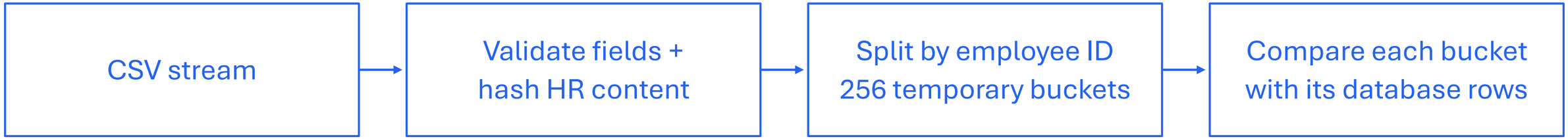


Delivery failure	Recovery	Booking safety
Exception / timeout	Backoff, then DEAD + one alert	Cancel future scheduled bookings only
Crash after consumer commit	Redelivery, dedupe by eventId	Rehire check keeps new bookings intact

Tests and implementation references are in the slide notes.

How the nightly comparison stays within memory

The service reads the CSV as a stream and compares a small part at a time.



Bound	Expected cost
Time	O(N + M)
Working heap	O((N + M) / B), plus fixed buffers
Temporary disk	O(N), B = 256

200k comparison: 16.388 s. Peak heap: 104.84 MiB (256 MiB cap).

Mixed-file median diff: 12.961 s. Its comparison path was warmed by the preceding exact-size case.

Tests and implementation references are in the slide notes.

How a snapshot repairs missed or incorrect state

The full nightly export can repair differences that an individual event must flag.

File / store comparison	Action or guard
No employee row	CREATE
Higher version, changed / same content	UPDATE / ADVANCE
Equal version, changed / same content	HEAL / UNCHANGED
Lower version	STALE_SKIP
Missing ACTIVE, hr_updated_at <= asOf	DEACTIVATE candidate
Missing ACTIVE, hr_updated_at > asOf	Keep employee
Present but invalid	Keep employee
Duplicate IDs in file	Highest version wins, equal-version clash invalid

Tests and implementation references are in the slide notes.

A short export waits for a human decision

A missing share of 30% or more stops the entire plan before any employee is changed.

Current console rehearsal, captured after rejecting the run

Safety brake at comparison

File appears to be missing 80,019 of 200,019 active employees (40.0%). Threshold is 30.0%. No one has been deactivated. Approve or reject this run.

Active before	Missing	Missing share	Threshold
200,019	80,019	40.0%	30.0%

Earlier recorded run: 79,591 / 198,995 missing (40.0%), 0 applied.

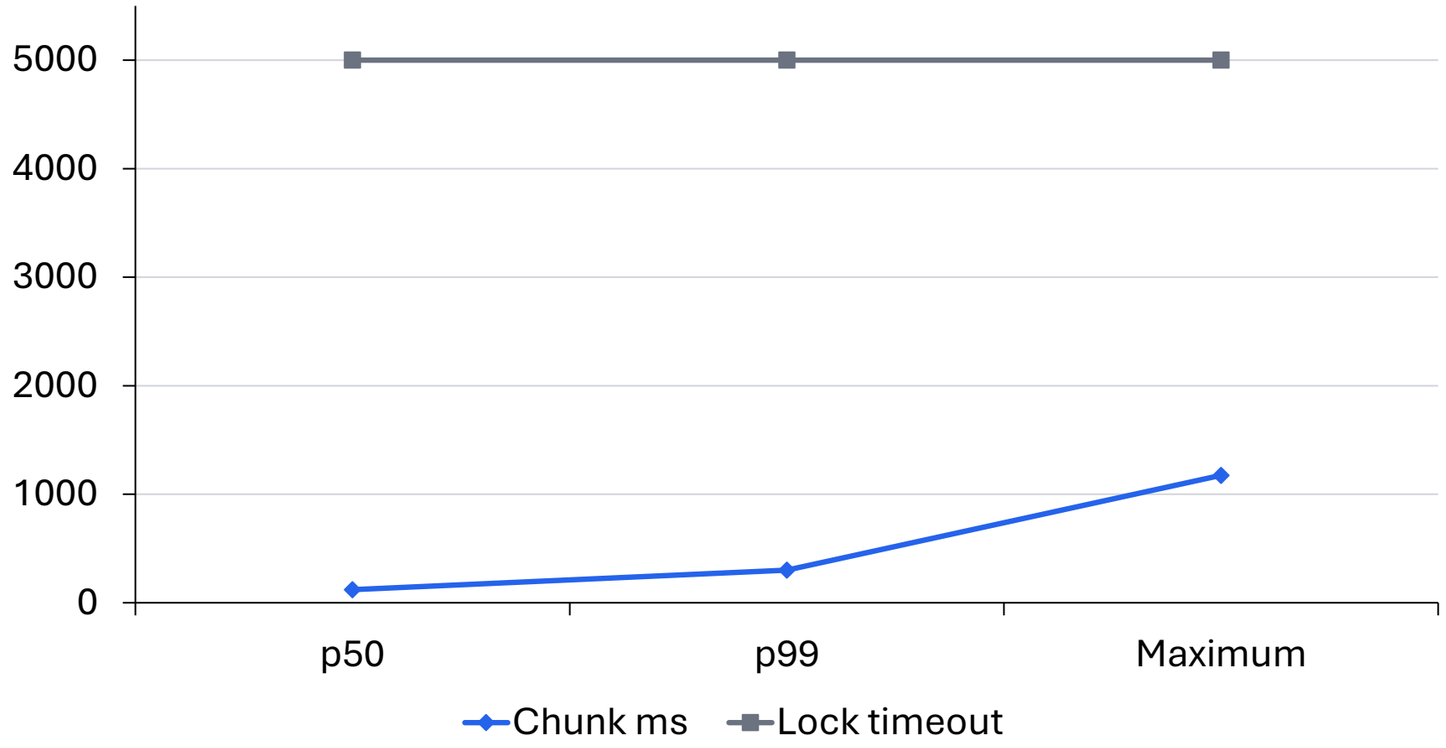


Dry run ends PLANNED. Reject ends REJECTED. Invalid or interrupted comparison ends FAILED.

Tests and implementation references are in the slide notes.

Small apply chunks keep other employees available

A chunk locks its selected employees. Events for other employees can continue.



237 committed chunks

Default size: 40

Median: 120.9 ms

99th percentile: 301.2 ms

Slowest: 1,174.4 ms

Employee changed meanwhile? Skip old action.

Restart? Resume pending actions.

p99 <1 s and max <2.5 s: pass.

Cold-JIT and host-load variation are retained, not discarded.

Tests and implementation references are in the slide notes.

Failures and recovery

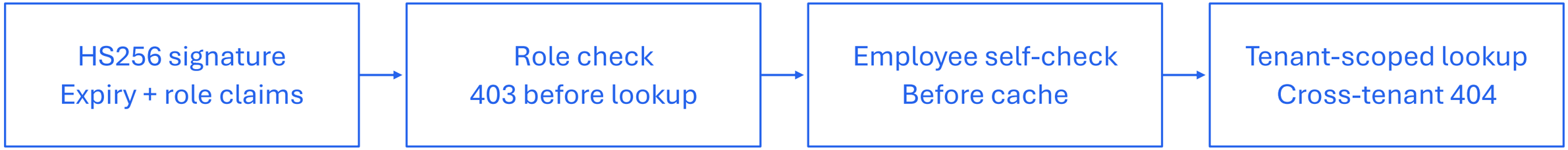
Each failure has a caller outcome, an operator signal and a recovery path.

Failure	Caller / operator sees	Recovery	Proof test
Duplicate	DUPLICATE	No employee mutation	ReplayIdempotencyIT
Out of order	STALE	Keep newer state	OutOfOrderIT
Crash after commit	Pending delivery	Restart publisher	OutboxCrashRecoveryIT
Crash in transaction	No partial state	Rollback, resend	TransactionAtomicityIT
Sink down	Retry, then DEAD	Backoff, alert, manual retry	OutboxRetryIT
Truncated export	AWAITING_APPROVAL	Human decision, zero apply	ReconciliationFlowIT
Concurrent events	Per-record outcomes	Row lock + max version	ConcurrentEventsIT
Database down	503 dependency error	Restore DB, resend	ErrorHandlingIT

Tests and implementation references are in the slide notes.

Who can access which data?

A valid token identifies the role and company. The server checks every request.



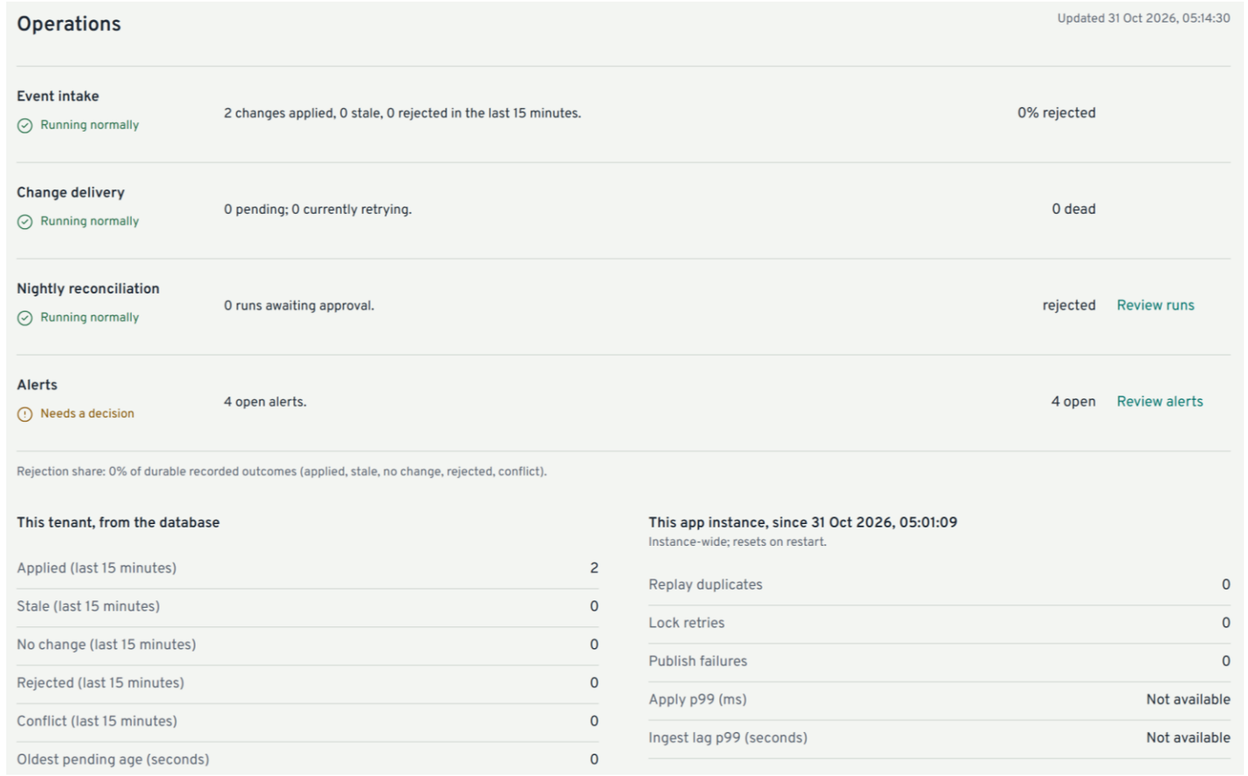
Boundary	Automated enforcement
Every protected handler	@PreAuthorize reflection + registered-route checks
Every tenant repository	Architecture guard requires UUID tenantId
No PII in operations	Canaries in logs, errors, outbox, rejections, alerts
Management	8081 loopback only, actuator absent from 8080

Console token stays in memory. Reload signs out. No browser token storage.

Tests and implementation references are in the slide notes.

What the operator can see

Database counts survive restarts. Process counters have an explicit restart boundary.



Operations

Updated 31 Oct 2026, 05:14:30

Event intake Running normally	2 changes applied, 0 stale, 0 rejected in the last 15 minutes.	0% rejected
Change delivery Running normally	0 pending; 0 currently retrying.	0 dead
Nightly reconciliation Running normally	0 runs awaiting approval.	rejected Review runs
Alerts Needs a decision	4 open alerts.	4 open Review alerts

Rejection share: 0% of durable recorded outcomes (applied, stale, no change, rejected, conflict).

This tenant, from the database

Applied (last 15 minutes)	2
Stale (last 15 minutes)	0
No change (last 15 minutes)	0
Rejected (last 15 minutes)	0
Conflict (last 15 minutes)	0
Oldest pending age (seconds)	0

This app instance, since 31 Oct 2026, 05:01:09

Instance-wide; resets on restart.

Replay duplicates	0
Lock retries	0
Publish failures	0
Apply p99 (ms)	Not available
Ingest lag p99 (seconds)	Not available

SQL provides durable tenant counts. Metrics provide rates and latency.

Summary section	Scope
durable	Tenant SQL, survives restart
sinceRestart	Instance-wide, reset on restart

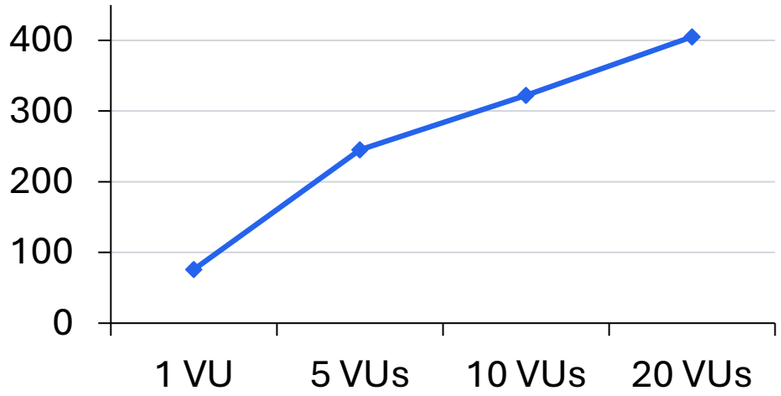
Cache	TTL / max
offices	10 min / 10,000
employeeView	30 s / 50,000
Write decisions	Never cached

Tests and implementation references are in the slide notes.

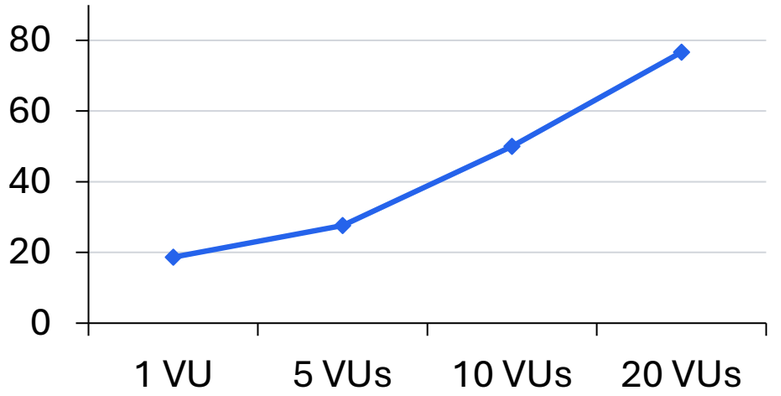
What I measured, and what those numbers mean

Workload, heap limit and latency definition stay visible beside the results.

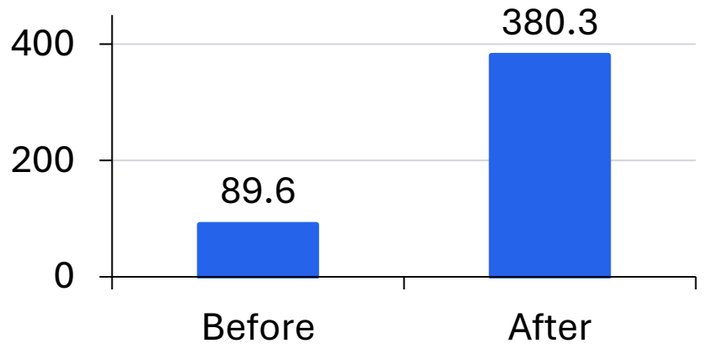
Batch 50: events / second



Amortized event p99 (ms)



Matched batching comparison



JDBC calls/action: 6.08 to 0.17

Latest local evidence	Value	Scope
Mixed diff / peak heap	12.961 s / 120.14 MiB	Median of 3, -Xmx256m
Chunk p50 / p99 / max	120.9 / 301.2 / 1,174.4 ms	40 actions, cold work included
Single-event p50 / p99	71.18 / 235.36 ms	Batch 1, 5 VUs, 60 s

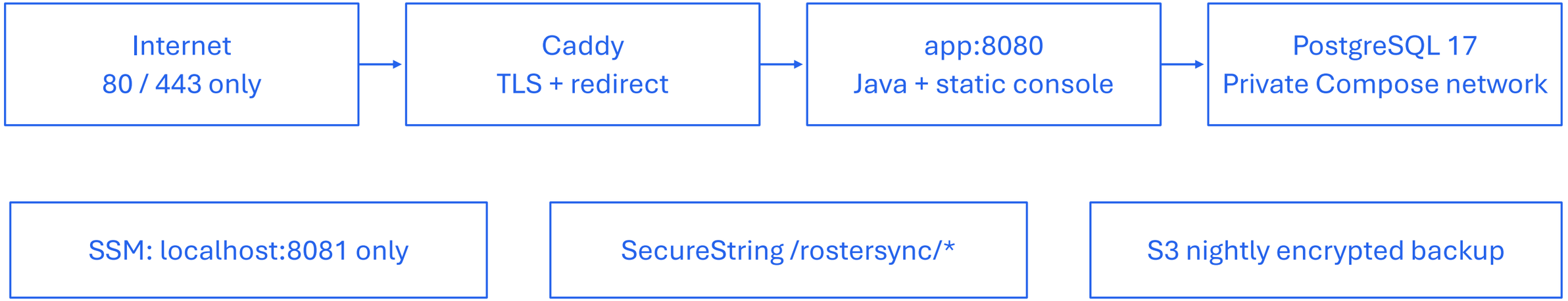
Above 5-10 VUs, commit I/O contributes and delivery falls behind. Pool 20 stays.

i5-1240P, 12 cores / 16 threads, 15.69 GiB RAM, Win11, Docker VM 7.60 GiB. Local, not AWS latency.

Tests and implementation references are in the slide notes.

The live deployment

The same service runs behind Caddy on one EC2 instance in Sydney.



Choice	Why / trade-off
EC2 + Compose	Same images, simple operations, one host to maintain
ECS Express Mode + RDS	Managed production path, extra services and cost
Lambda	Poor fit for this long-lived worker and poller

Sydney, t4g.small. Estimate ~$21.15/month. Explicit IAM Deny blocks parameter reads outside /rostersync/*.

Tests and implementation references are in the slide notes.

Trade-offs in this implementation

The design prioritises correct writes and explainable recovery on one host.

Choice	What it protects	What it costs
Fail closed without PostgreSQL	Ordered durable writes	Temporary ingestion unavailability
One transaction per event	Per-record isolation and result	Large-batch response latency
At-least-once + dedupe	No lost committed change	Idempotent consumer work
Local Caffeine	Fast bounded reads	TTL staleness across instances
Single app / worker host	Simple ownership and recovery	No high-availability promise

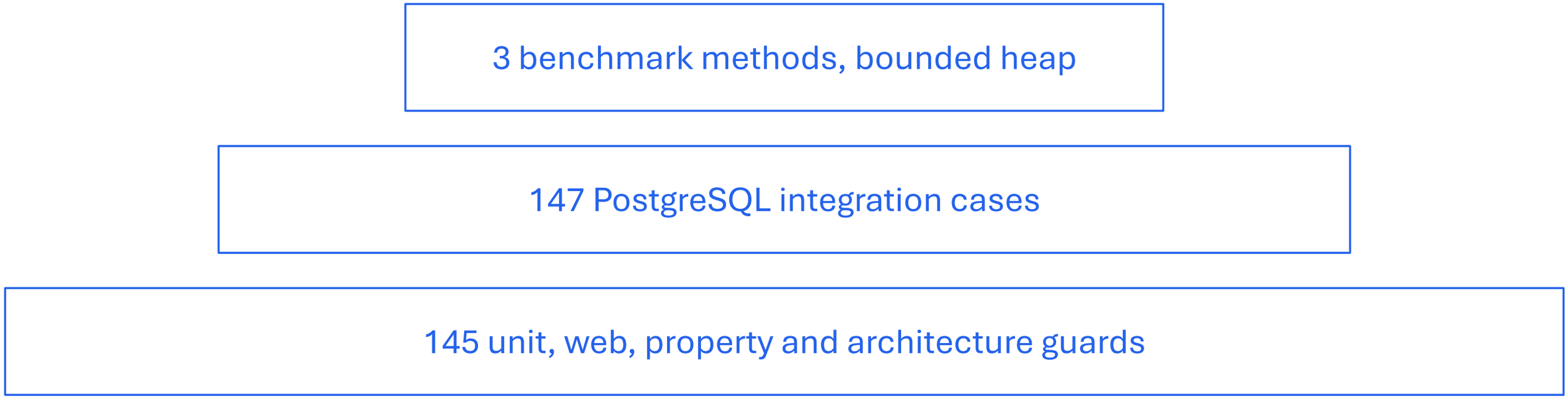
Not included: a message broker, shared cache or external identity provider.

Also deferred: database row policies and cancellation of active runs.

Tests and implementation references are in the slide notes.

How I tested the hard rules

Tests exercise decisions, concurrent writes, rollback and recovery after a restart.



Domain line coverage: 359 / 361 = 99.45%

26 frontend unit cases

17 local + 6 earlier live browser cases

56 / 57 case-study rows done

1 partial: demo video

96 local/live axe scans, no serious/critical findings

Tests and implementation references are in the slide notes.

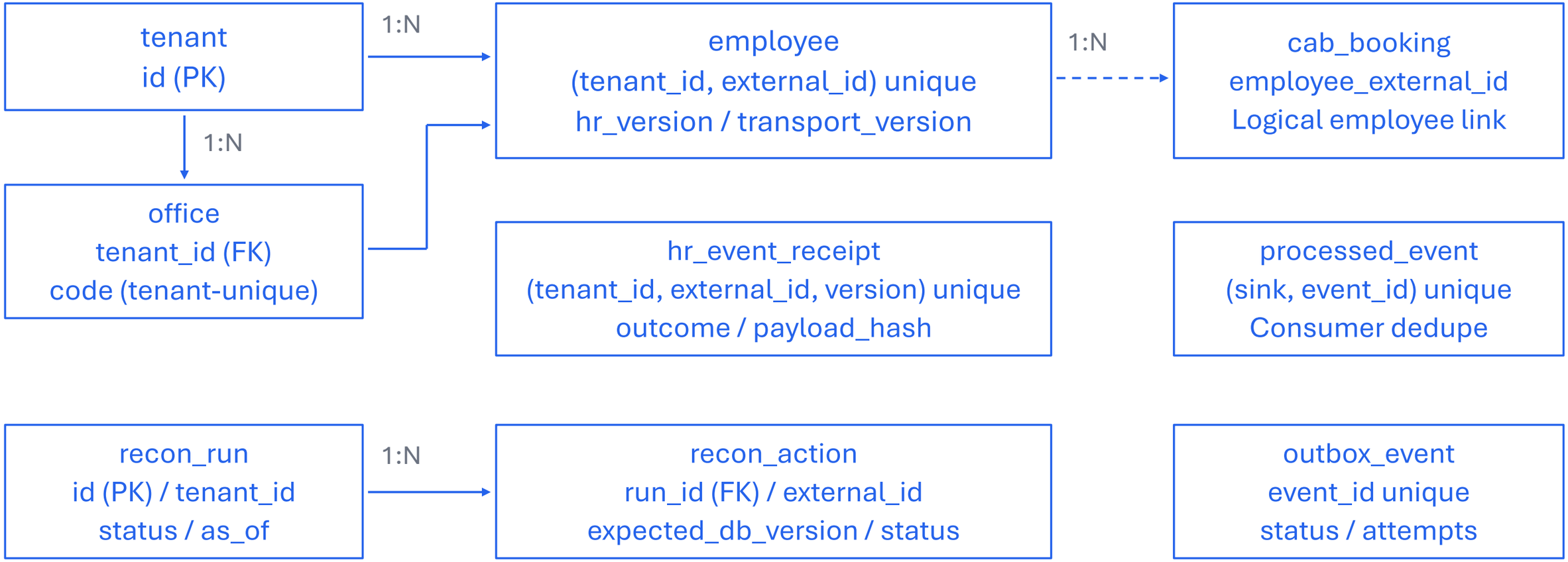
What I would change as usage grows

The next steps follow the limits measured in this implementation.

Pressure	Next change	Invariant to retain
More downstream consumers	Outbox relay to Kafka / CDC	Employee + outbox atomicity
Larger event history	Time-partition receipt and outbox tables	Stable retention and replay policy
More application instances	Durable leases + S3 uploads	One active reconciliation owner
Shared hot reads	Redis or invalidation events	Tenant key + bounded staleness
Stronger tenant defense	PostgreSQL RLS	Tenant-scoped API behavior

The next performance limit to measure is sustained delivery, alongside ingestion bursts.

Database tables and their responsibilities



Solid: database foreign key. Dashed: logical employee link. Other tenant IDs are scoped in queries.

How a request reaches a controller



HS256, one-hour TTL, 30-second clock allowance. No DB lookup per token validation.

Role check first. Tenant-scoped lookup second. EMPLOYEE self-check before cache.

Finally clears ThreadLocal and MDC, including error paths.

Tests and implementation references are in the slide notes.

Which employee rows does a chunk lock?

```
select ee1_0.id, ee1_0.created_at, ee1_0.deactivated_at,
       ee1_0.deactivation_source, ee1_0.email, ee1_0.external_id,
       ee1_0.hr_updated_at, ee1_0.hr_version, ee1_0.manager_external_id,
       ee1_0.name, ee1_0.office_id, ee1_0.phone, ee1_0.pickup_point,
       ee1_0.recon_bucket, ee1_0.shift_code, ee1_0.special_assistance,
       ee1_0.status, ee1_0.tenant_id, ee1_0.transport_updated_at,
       ee1_0.transport_version, ee1_0.updated_at
from employee ee1_0
where ee1_0.tenant_id=?
  and ee1_0.external_id in (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,
                            ?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)
order by ee1_0.external_id
for no key update of ee1_0
```

40 ID placeholders. IN has the same employee-set scope as external_id = ANY(?).

Tests and implementation references are in the slide notes.

Request latency versus batch cost per event

Batch / VUs	Events/s	Request p50 / p99 ms	Event p50 / p99 ms	Pool pending	PG commits/s
1 / 5	59.0	71.2 / 235.4	71.18 / 235.36	0	61.2
50 / 1	75.9	650.6 / 933.1	13.01 / 18.66	0	96.2
50 / 5	245.1	985.6 / 1,380.6	19.71 / 27.61	0	243.4
50 / 10	322.1	1,446.4 / 2,502.7	28.93 / 50.05	0	301.2
50 / 20	405.0	2,325.4 / 3,832.8	46.51 / 76.66	1	372.9
50 / 20, sync off	492.0	1,873.7 / 3,780.7	37.47 / 75.61	2	436.0

Batch 50: event cost = request duration / 50. Batch 1 is actual response latency.

All cases: 60 s, zero failures. Last row is diagnosis only, durability restored to on.

Pool stays 20. Highest normal run ended with 22,260 pending outbox rows.

How the safety brake checks its threshold

missingActive >= maxMissingRatio × activeBefore

Inputs	Result
2,999 missing of 10,000 active	29.99%: pass
3,000 missing of 10,000 active	30.00%: trip
Zero active employees	Skip ratio/count only
Trailer or expected row count mismatch	Trip, even with zero active

Reason order: ROW_COUNT_MISMATCH, MISSING_RATIO, MISSING_COUNT.

Locale.ENGLISH formats thousands and one-decimal percentages.

Tests and implementation references are in the slide notes.

Why there are two hashes

	CRC32	SHA-256
Purpose	Stable bucket assignment	Canonical content comparison
Input	Employee external ID	Owned HR state, fixed field order
Output use	Unsigned CRC32 modulo 256	Record hash / receipt payload hash
Security claim	None	Collision-resistant content digest
Receipt detail	Not part of receipt hash	Event type included, eventId/time excluded

Neither hash makes a business decision by itself. Version and ownership rules still apply.

Tests and implementation references are in the slide notes.

Demo access and production boundaries

Mode	Behavior
prod	No demo seeding, dev endpoints or HALT
prod,demo on this live server	OPERATOR-only tools, one-shot faults
Public browser build	Account help, no password, memory-only token
Credential delivery	Private SSM retrieval, never README or logs
Operational access	SSM, management loopback, public 80/443 only

Tests and implementation references are in the slide notes.